CSC2302 DATA STRUCTURES COMPLETE STUDY GUIDE

Simple English - Step by Step - With Examples

For students who need help understanding Python

Covers: Python Review | Linked Lists | Stacks | Queues
Trees | BST | Big O | Sorting | Searching | Graphs
Exam Format | Assignments | Practice Problems

Al Akhawayn University in Ifrane

School of Science and Engineering

Based on the CSC2302 course materials

TABLE OF CONTENTS

PART 0: Exam Format - What is on the exam

PART 1: Python Review - variables, print, input, if, loops, lists, dicts, files

PART 2: Big O Notation - how fast is your code

PART 3: Searching and Sorting - find and organize data

PART 4: Linked Lists - chain of nodes

PART 5: Stacks - Last In, First Out

PART 6: Queues - First In, First Out

PART 7: Trees and Binary Search Trees

PART 8: Graphs - nodes and connections

PART 9: Recursion - function that calls itself

PART 10: Assignments - BST Inventory, Student Data, Queue/Stack exercises

PART 11: Practice Problems with Solutions

PART 12: Quick Reference Sheets

PART 0 - EXAM FORMAT

This part is IMPORTANT. Read this first. It tells you exactly what is on the exam.

MIDTERM EXAM (Oct 23, 8:30-10:30)

10 exercises, total 100 points:
Part 1: Python Data Structures (20 points - 2 exercises)
Coding exercises - you write function definitions
Part 2: Searching and Sorting (20 points - 2 exercises)
Coding exercises - you write function definitions
Part 3: Big O Notation (20 points - 2 exercises)
NO CODING - just analysis/written answers
Part 4: Linked Lists (40 points - 4 exercises)
Coding exercises - you write function definitions

FINAL EXAM

4 parts:
Part 1: Trees (30 points)
Part 2: Heaps (30 points)
Part 3: Hash Tables (40 points)
Part 4: Graphs (10 points bonus, no coding)

Exam Rules (MEMORIZE THESE)

1. No phones. Turn off and put in bag.
2. No calculators or smart watches.
3. You get 4 pages scratch paper. Use it well.
4. No questions during exam. Read instructions carefully.
5. Use descriptive names: **gpa** not **x**
6. Plagiarism = WF grade + report to disciplinary committee.
7. Leave your AUI ID on desk.
8. No leaving room unless you submit your exam.

What the exam covers (from Canvas):

- Chapter 2 & 3: Sorting, Searching, Algorithm Analysis
- Lab Sessions: Linked List style problems
- Python: lists, tuples, dictionaries (may be combined with Ch 2, 3, 4)
- Case studies on Linked Lists

IMPORTANT: How to define functions/methods:

If the problem says 'instance method' -> define INSIDE a class, first parameter = self
If the problem says 'function' -> do NOT use self
If file is already open -> do NOT open it again, accept file handler as parameter
If object already created -> do NOT create it, accept as parameter

PART 1 - PYTHON REVIEW

This part is from the **Python Programming Review** presentation.

If you already know Python basics, skip to Part 4 (Linked Lists).

1.1 Variables

A **variable** is a name for a value stored in memory. Think: box with a label.

Python **infers** the type automatically (you don't need to say 'this is an integer').

```
x = 10 # x is int (whole number)
y = 10.5 # y is float (decimal)
name = "Ahmed" # str (text)
is_student = True # bool (True/False)

# Multiple variables at once:
a, b, c = 1, 2, 3
```

Variable Rules:

- Start with letter or _ (underscore)

- No spaces. Use _ instead.

- Case sensitive: **name** and **Name** are DIFFERENT

- Cannot use Python keywords: **def, class, if, for, while** etc.

1.2 Arithmetic Operations

```
a = 10
b = 3
print(a + b) # 13 (addition)
print(a - b) # 7 (subtraction)
print(a * b) # 30 (multiplication)
print(a / b) # 3.333... (division - always gives float)
print(a // b) # 3 (floor division - integer result)
print(a % b) # 1 (modulus - remainder)
print(a ** b) # 1000 (exponent - 10 to the power 3)

# Shorthand:
a += 5 # same as: a = a + 5
a -= 3 # same as: a = a - 3
```

1.3 Input and Output (from slides)

```
# OUTPUT - print()
print("Hello") # basic
print("Age:", 20) # multiple items, separated by space
print("Age: " + str(20)) # using + (no space added)

# BEST: f-string (formatted string)
name = "Ahmed"
age = 20
print(f"My name is {name} and I am {age}")
# shows: My name is Ahmed and I am 20

# INPUT - get text from user
name = input("Enter your name: ") # returns STRING always

# Multiple inputs on one line:
x, y = input("Enter two numbers: ").split()
# User types: "10 20" -> x="10", y="20"
# Convert to int: x = int(x)
```

1.4 If Statements (selections)

```
score = 85
if score >= 90:
grade = "A"
elif score >= 80:
grade = "B"
elif score >= 70:
grade = "C"
else:
grade = "F"

# Comparison operators:
# == equal != not equal
```

```
# > greater than < less than
# >= >= <= <=

# Logical operators (combine conditions):
# and BOTH True or at least ONE True not reverse
```

1.5 Loops (repetitions)

```
# FOR LOOP - iterate over a sequence
for i in range(5): # 0, 1, 2, 3, 4
print(i)

for i in range(2, 6): # 2, 3, 4, 5
print(i)

for fruit in ["apple", "banana", "cherry"]:
print(fruit)

# WHILE LOOP - keep going while condition is True
count = 0
while count < 5:
print(count)
count += 1 # IMPORTANT: update or loop runs forever!

# Loop control:
# break - exit loop immediately
# continue - skip to next iteration
# pass - do nothing (placeholder)
```

1.6 Lists (from Python Review slides)

A list is a collection of items. Ordered, changeable, allows duplicates.

```
nums = [1, 2, 3, 4, 5]

# Access: nums[0] -> 1, nums[-1] -> 5
# Change: nums[0] = 10
# Add to end: nums.append(6)
# Remove by value: nums.remove(3)
# Sort: nums.sort()
# Length: len(nums)

# Slicing (take part of a list):
nums[1:3] # [2, 3] (from index 1 to 2, 3 is NOT included)
nums[:3] # [1, 2, 3]
nums[2:] # [3, 4, 5]

# Matrix (list of lists):
matrix = [
[1, 2, 3],
[4, 5, 6],
[7, 8, 9]
]
print(matrix[0][1]) # 2 (row 0, column 1)
```

1.7 Tuples

A tuple is like a list but **immutable** (cannot change after creation).

```
t = (1, 2, 3)
print(t[0]) # 1
# t[0] = 10 ERROR! Cannot change tuple
```

1.8 Dictionaries

A dictionary stores **key-value pairs**. Like a phone book: name -> number.

```
student = {
"name": "Ahmed",
"age": 20,
"major": "CS"
}
```

```
print(student["name"]) # Ahmed
print(student.get("name")) # Ahmed (safer)
student["age"] = 21 # change value
student["city"] = "Ifrane" # add new key

# Loop through:
for key in student:
print(key, student[key])

for key, val in student.items():
print(f"{key}: {val}")

# Count occurrences (common exam question):
nums = [2, 2, 4, 5, 4, 4, 5, 7]
counts = {}
for n in nums:
if n in counts:
counts[n] += 1
else:
counts[n] = 1
print(counts) # {2: 2, 4: 3, 5: 2, 7: 1}
```

1.9 Strings

```
s = "Hello World"
len(s) # 11
s.upper() # "HELLO WORLD"
s.lower() # "hello world"
s.find("World") # 6 (position where "World" starts)
s.count("l") # 3 (how many 'l'?)
s.split(" ") # ["Hello", "World"]
" ".join(["a","b"]) # "a b"
s[0:5] # "Hello" (slicing works like lists)

# Check if substring exists:
if "World" in s:
print("Found!")
```

1.10 Functions

```
def add(a, b):
return a + b

result = add(3, 5) # result = 8

# Default parameter values:
def greet(name="Student"):
print(f"Hello {name}")

# *args = any number of arguments
def sum_all(*args):
return sum(args)

print(sum_all(1, 2, 3, 4)) # 10

# lambda = short anonymous function
double = lambda x: x * 2
print(double(5)) # 10

# map, filter, sorted (built-in functions):
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x*2, nums)) # [2, 4, 6, 8]
evens = list(filter(lambda x: x%2==0, nums)) # [2, 4]
s = sorted(nums, reverse=True) # [4, 3, 2, 1]
```

1.11 File I/O

To read or write files, use **open()**. ALWAYS close the file or use **with**.

```
# Modes: r = read, w = write (overwrites!), a = append
# b = binary (for images/videos)

# Read entire file:
```

```
with open("file.txt", "r") as f:
content = f.read()

# Read all lines into a list:
with open("file.txt", "r") as f:
lines = f.readlines()

# Read line by line:
with open("file.txt", "r") as f:
for line in f:
print(line.strip())

# Write:
with open("output.txt", "w") as f:
f.write("Hello\n")
f.write("World\n")

# seek() - move cursor to a position:
with open("file.txt", "r") as f:
content = f.read() # read everything
f.seek(0) # go back to start
first_10 = f.read(10) # read first 10 characters
```

PART 2 - BIG O NOTATION

Big O tells you how runtime grows as input size (n) gets bigger.

Source: Course slides + Exam Part 3 (20 points, NO CODING).

2.1 The Big O Classes

```
O(1) = CONSTANT - same time no matter what
Example: get arr[0], push to stack
O(log n) = LOGARITHMIC - grows very slowly
Example: Binary Search, BST search
O(n) = LINEAR - grows with n
Example: loop through a list
O(n log n) = LINEATHM - sorting algorithms
Example: Merge Sort, Quick Sort
O(n^2) = QUADRATIC - nested loops
Example: Bubble Sort, nested for
```

2.2 How to Calculate

```
# 1. Drop constants:
# O(2n) -> O(n) O(500) -> O(1) O(n/2) -> O(n)

# 2. Keep the biggest term only:
# O(n + n^2) -> O(n^2)
# O(n + log n) -> O(n)
# O(1 + n + n^2) -> O(n^2)

# 3. Sequential operations add, nested loops multiply:
# for i in range(n): O(n)
# for j in range(n): O(n * n) = O(n^2)
# print(i, j)
```

2.3 Complexity of Data Structures (MEMORIZE)

```
Structure Search Insert Delete
Array/List O(n) O(n) O(n)
Linked List O(n) O(1)* O(1)*
Stack - O(1) O(1)
Queue - O(1) O(1)
BST (balanced) O(log n) O(log n) O(log n)
BST (worst) O(n) O(n) O(n)

*Linked List: insert/delete at head is O(1), at end/tail is O(n) without tail ptr
*Stack: push/pop = O(1)
*Queue: enqueue/dequeue = O(1) with deque
```

PART 3 - SEARCHING AND SORTING

Source: Course slides. Exam: Part 2 = 20 points.

3.1 Linear Search - O(n)

```
def linear_search(arr, target):
for i in range(len(arr)):
if arr[i] == target:
return i # FOUND, return index
return -1 # NOT found

# Checks every element one by one.
# Simple but SLOW for big lists.
```

3.2 Binary Search - O(log n)

REQUIREMENT: List MUST be sorted first!

```
def binary_search(arr, target):
low = 0
high = len(arr) - 1

while low <= high:
mid = (low + high) // 2 # find middle

if arr[mid] == target:
return mid # FOUND!
elif arr[mid] < target:
low = mid + 1 # target in right half
else:
high = mid - 1 # target in left half
return -1 # NOT found

# Trace: binary_search([2, 5, 8, 12, 16, 23], 16)
# low=0 high=5 mid=2 arr[2]=8 -> 8<16, go RIGHT
# low=3 high=5 mid=4 arr[4]=16 -> FOUND! (2 steps)
# Linear search would take 5 steps!
```

3.3 Bubble Sort - O(n^2)

```
def bubble_sort(arr):
n = len(arr)
for i in range(n-1):
for j in range(n-1-i):
if arr[j] > arr[j+1]:
arr[j], arr[j+1] = arr[j+1], arr[j]
return arr

# Each pass: largest element "bubbles up" to correct position
# [5, 3, 8, 1] -> pass 1: [3,5,1,8] -> pass 2: [3,1,5,8] -> [1,3,5,8]
```

3.4 Selection Sort - O(n^2)

```
def selection_sort(arr):
n = len(arr)
for i in range(n):
min_idx = i
for j in range(i+1, n):
if arr[j] < arr[min_idx]:
min_idx = j
arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr

# Find smallest, put at position 0.
# Find next smallest, put at position 1. Etc.
```

3.5 Insertion Sort - O(n^2) but fast for nearly-sorted data

```
def insertion_sort(arr):
for i in range(1, len(arr)):
key = arr[i]
j = i - 1
while j >= 0 and arr[j] > key:
arr[j + 1] = arr[j]
j -= 1
arr[j + 1] = key
```

```
return arr

# Like sorting cards in your hand.
# Take one card, insert in correct position.
```

PART 4 - LINKED LISTS

EXAM WEIGHT: Part 4 = 40 points (4 exercises)

Sources: c.pptx (Linked Lists module), Python Review slides, BST file.

A **linked list** is a chain of **nodes**. Each node has data and a pointer to the next node.

Unlike Python lists, linked list elements are NOT stored next to each other in memory.

They are connected by pointers (like a treasure hunt where each clue tells you where the next clue is).

4.1 What is a Node?

```
class Node:
def __init__(self, data):
self.data = data # the value
self.next = None # pointer to next node

# Visual:
# [data | next] -> [data | next] -> [data | None]
```

4.2 Singly Linked List Class

```
class LinkedList:
def __init__(self):
self.head = None # first node
self.tail = None # last node

def append(self, data):
new_node = Node(data)
if self.head is None:
self.head = new_node
self.tail = new_node
else:
self.tail.next = new_node
self.tail = new_node

def display(self):
curr = self.head
while curr:
print(curr.data, end=" -> ")
curr = curr.next
print("None")

def __str__(self):
result = []
curr = self.head
while curr:
result.append(str(curr.data))
curr = curr.next
return " -> ".join(result)

def __len__(self):
count = 0
curr = self.head
while curr:
count += 1
curr = curr.next
return count
```

4.3 Remove a node by value

```
def remove(self, target):
curr = self.head
prev = None
while curr:
if curr.data == target:
if prev is None:
self.head = curr.next
else:
prev.next = curr.next
if curr == self.tail:
self.tail = prev
return True
prev = curr
curr = curr.next
return False
```

4.4 Pop (remove at index)

```
def pop(self, index):
if index < 0 or index >= len(self):
raise IndexError("Index out of range")
curr = self.head
prev = None
```

```
for _ in range(index):
prev = curr
curr = curr.next
if prev is None:
self.head = curr.next
else:
prev.next = curr.next
if curr == self.tail:
self.tail = prev
return curr.data
```

4.5 Insert at Position

```
def insert(self, index, data):
new_node = Node(data)
if index == 0:
new_node.next = self.head
self.head = new_node
if self.tail is None:
self.tail = new_node
return
curr = self.head
for _ in range(index - 1):
curr = curr.next
new_node.next = curr.next
curr.next = new_node
if new_node.next is None:
self.tail = new_node
```

4.6 Reverse Linked List (in-place)

```
def reverse(self):
prev = None
curr = self.head
self.tail = curr
while curr:
next_node = curr.next
curr.next = prev
prev = curr
curr = next_node
self.head = prev

# 10 -> 20 -> 30 -> None
# Step 1: curr=10, prev=None, save 20, 10.next=None
# Step 2: curr=20, prev=10, save 30, 20.next=10
# Step 3: curr=30, prev=20, save None, 30.next=20
# Result: 30 -> 20 -> 10 -> None
```

4.7 Find Middle Node (Two-Pointer/Fast & Slow)

```
def find_middle(self):
slow = self.head
fast = self.head
while fast and fast.next:
slow = slow.next
fast = fast.next.next
return slow.data

# [10 -> 20 -> 30 -> 40 -> 50]
# slow: [10] [20] [30]
# fast: [10] [30] [50] (stops)
# Middle: 30
```

4.8 Remove Duplicates

```
def remove_duplicates(self):
seen = set()
curr = self.head
prev = None
while curr:
if curr.data in seen:
prev.next = curr.next
```

```
if curr == self.tail:
self.tail = prev
else:
seen.add(curr.data)
prev = curr
curr = curr.next
```

4.9 Merge Two Sorted Linked Lists

```
@staticmethod
def merge_sorted(list1, list2):
dummy = Node(0)
tail = dummy
c1, c2 = list1.head, list2.head
while c1 and c2:
if c1.data <= c2.data:
tail.next = c1
c1 = c1.next
else:
tail.next = c2
c2 = c2.next
tail = tail.next
tail.next = c1 if c1 else c2
return dummy.next

# List1: 1 -> 5 -> 9
# List2: 3 -> 6 -> 7
# Result: 1 -> 3 -> 5 -> 6 -> 7 -> 9
```

4.10 Doubly Linked List (extra)

```
class DoublyNode:
def __init__(self, data):
self.data = data
self.next = None
self.prev = None

# None <- [prev|data|next] <-> [prev|data|next] -> None
```

4.11 Linked List vs Python List

Use Linked List WHEN: frequent insert/delete at beginning, unknown size, dynamic data

Use Python List WHEN: need fast random access by index (O(1) lookup)

```
Feature Python List Linked List
Memory Contiguous Non-contiguous
Random access O(1) O(n)
Insert at head O(n) O(1)
Insert at tail O(1) amortized O(1) (with tail)
Delete at head O(n) O(1)
Space overhead Low Extra pointer per node
```

PART 5 - STACKS

LIFO = Last In, First Out. Like a stack of plates.

Sources: Stacks & Queues slides, Stacks Case Study, Queues & Stacks Assignment.

5.1 Stack Operations

```
push(value) # add to top O(1)
pop() # remove from top O(1)
peek() # see top without removing O(1)
is_empty() # check if empty O(1)
```

5.2 Stack - List Implementation

```
class Stack:
def __init__(self):
self.__L = []
def push(self, value):
self.__L.append(value)
def pop(self):
if self.is_empty():
raise ValueError("Stack is empty")
return self.__L.pop()
def peek(self):
if self.is_empty():
raise ValueError("Stack is empty")
return self.__L[-1]
def is_empty(self):
return len(self.__L) == 0

s = Stack()
s.push(10); s.push(20); s.push(30)
s.pop() # 30
s.peek() # 20
```

5.3 Check Brackets - ( ) [ ] { }

```
def is_valid_brackets(s):
stack = []
pairs = {")": "(", "}": "{", "]": "["}
for char in s:
if char in "({[":
stack.append(char)
elif char in ")}]":
if not stack or stack[-1] != pairs[char]:
return False
stack.pop()
return len(stack) == 0

# "({[]})" -> True "(]" -> False
```

5.4 Browser History (Two Stacks)

```
class BrowserHistory:
def __init__(self):
self.back = []
self.forward = []
self.current = None
def visit(self, url):
if self.current:
self.back.append(self.current)
self.current = url
self.forward.clear()
def back(self):
if not self.back:
return False
self.forward.append(self.current)
self.current = self.back.pop()
return True
def forward(self):
if not self.forward:
return False
self.back.append(self.current)
self.current = self.forward.pop()
return True
```

5.5 Reverse String Using Stack

```
def reverse_string(s):
stack = []
for char in s:
stack.append(char)
result = ""
while stack:
result += stack.pop()
return result

# hello -> push h,e,l,l,o -> pop o,l,l,e,h -> olleh
```

PART 6 - QUEUES

FIFO = First In, First Out. Like a line at a store.

Sources: Queues Case Study, Stacks & Queues slides, Queues & Stacks Assignment.

6.1 Queue Operations

```
enqueue(value) # add to back O(1)
dequeue() # remove from front O(1)
front() # see front without removing O(1)
```

6.2 Queue using deque (BEST)

```
from collections import deque

class Queue:
def __init__(self):
self.__L = deque()
def enqueue(self, value):
self.__L.append(value)
def dequeue(self):
if self.is_empty():
raise ValueError("Queue is empty")
return self.__L.popleft()
def front(self):
if self.is_empty():
raise ValueError("Queue is empty")
return self.__L[0]
def is_empty(self):
return len(self.__L) == 0

q = Queue()
q.enqueue(10); q.enqueue(20); q.enqueue(30)
q.dequeue() # 10 (FIRST in, FIRST out)
```

6.3 Deque - Double-Ended Queue

```
from collections import deque
d = deque()
d.append(10) # add right [10]
d.appendleft(5) # add left [5, 10]
d.popleft() -> 5 # remove left [10, 20]
d.pop() -> 20 # remove right [10]
```

6.4 Airport Security Queue

```
class SecurityQueue:
def __init__(self):
self.queue = deque()
def enqueue(self, name):
self.queue.append(name)
def dequeue(self):
return self.queue.popleft() if self.queue else None

def process_events(filepath):
q = SecurityQueue()
served = []
with open(filepath) as f:
for line in f:
parts = line.strip().split()
if parts[0] == "ARRIVE":
q.enqueue(parts[1])
elif parts[0] == "SERVE":
name = q.dequeue()
if name:
served.append(name)
return served
```

6.5 Reverse Queue Using Stack

```
def reverse_queue(q):
stack = []
while q:
stack.append(q.popleft())
while stack:
q.append(stack.pop())
return q
```

```
# [10, 20, 30, 40, 50] -> [50, 40, 30, 20, 10]
```

6.6 Filter Odd Numbers

```
def filter_odds(q):
result = deque()
while q:
val = q.popleft()
if val % 2 == 1:
result.append(val)
return result

# [12, 7, 4, 15, 8, 21, 6, 9] -> [7, 15, 21, 9]
```

6.7 Merge Two Sorted Queues

```
def merge_sorted_queues(q1, q2):
result = deque()
while q1 and q2:
if q1[0] <= q2[0]:
result.append(q1.popleft())
else:
result.append(q2.popleft())
while q1: result.append(q1.popleft())
while q2: result.append(q2.popleft())
return result

# [1,5,9,12,18] + [3,6,7,15,20,25] -> [1,3,5,6,7,9,12,15,18,20,25]
```

6.8 Merge Two Stacks Alternately

```
def merge_stacks_alternate(s1, s2):
result = []
while s1 and s2:
result.append(s1.pop())
result.append(s2.pop())
while s1: result.append(s1.pop())
while s2: result.append(s2.pop())
return result

# S1: [5,10,15] S2: [A,B,C,D]
# Result: [5,A,10,B,15,C,D]
```

PART 7 - TREES AND BINARY SEARCH TREES

Final exam: Trees = 30 points

Sources: Tree_Data_Structures_Intro.pptx, Binary_Trees_Recursion_Practice.pdf,

BST implementation file, BST Inventory Assignment.

7.1 What is a Tree?

A **tree** is a hierarchical structure. Like a family tree or company hierarchy.

- **Root** = top node (no parent)

- **Child** = node connected below another

- **Leaf** = node with no children

- **Edge** = connection between two nodes

- **Height** = edges from root to deepest leaf

Tree rules:

1. Connected - path between any two nodes

2. Acyclic - no cycles (loops)

3. n nodes = n-1 edges

7.2 Types of Trees

```
Binary Tree: each node has at most 2 children
Ternary Tree: each node has at most 3 children
N-ary Tree: any number of children
Complete Tree: all levels full except possibly last, left-filled
Full/Perfect: all levels completely filled
Skewed Tree: all nodes have one child (like linked list)
Balanced Tree: height diff between left/right <= 1
BST: left < node < right
Heap: parent smaller (min-heap) or larger (max-heap) than children
```

7.3 TreeNode

```
class TreeNode:
def __init__(self, value):
self.value = value
self.left = None
self.right = None

# Tree:
# 1
# / \
# 2 3
# /\ /
# 4 5 6

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)
```

7.4 Tree Traversals

```
# INORDER: Left -> Node -> Right (BST -> SORTED!)
def inorder(root):
if root:
inorder(root.left)
print(root.value, end=" ")
inorder(root.right)

# PREORDER: Node -> Left -> Right
def preorder(root):
if root:
print(root.value, end=" ")
preorder(root.left)
preorder(root.right)

# POSTORDER: Left -> Right -> Node
def postorder(root):
if root:
```

```
postorder(root.left)
postorder(root.right)
print(root.value, end=" ")

# LEVEL-ORDER (BFS using queue)
from collections import deque
def level_order(root):
if not root:
return
q = deque([root])
while q:
node = q.popleft()
print(node.value, end=" ")
if node.left: q.append(node.left)
if node.right: q.append(node.right)

# For tree [1, 2, 3, 4, 5, 6]:
# Inorder: 4 2 5 1 6 3
# Preorder: 1 2 4 5 3 6
# Postorder: 4 5 2 6 3 1
```

7.5 BST Rule (MEMORIZE THIS)

For ANY node: LEFT subtree values are SMALLER. RIGHT subtree values are BIGGER.

```
# 50
# / \
# 30 70
# / \ / \
# 20 40 60 80
#
# Inorder: 20 30 40 50 60 70 80 (SORTED!)
```

7.6 BST - Full Implementation

```
class Node:
def __init__(self, value):
self.value = value
self.left = None
self.right = None

class BST:
def __init__(self):
self.root = None

def insert(self, root, value):
if value <= root.value:
if root.left is None:
root.left = Node(value)
else:
self.insert(root.left, value)
else:
if root.right is None:
root.right = Node(value)
else:
self.insert(root.right, value)

def append(self, value):
if self.root is None:
self.root = Node(value)
else:
self.insert(self.root, value)

def search(self, target):
def _search(root, target):
if root is None: return False
if root.value == target: return True
elif target < root.value:
return _search(root.left, target)
else:
return _search(root.right, target)
return _search(self.root, target)
```

7.7 BST Remove - 3 Cases

Case 1: Leaf (no children) - just delete

Case 2: One Child - bypass the node

Case 3: Two Children - find in-order successor (go right once, then left as far as possible)

```
def remove(self, target):
p_curr, curr = None, self.root
while curr:
if curr.value == target:
break
elif target < curr.value:
p_curr, curr = curr, curr.left
else:
p_curr, curr = curr, curr.right
if curr is None:
raise ValueError("Target not found!")

# CASE 3: Two children
if curr.left and curr.right:
p_succ = curr
succ = curr.right
while succ.left:
p_succ, succ = succ, succ.left
curr.value = succ.value
if p_succ.left is succ:
p_succ.left = succ.right
else:
p_succ.right = succ.right
# CASES 1 & 2: 0 or 1 child
else:
child = curr.left or curr.right
if p_curr is None:
self.root = child
elif p_curr.left is curr:
p_curr.left = child
else:
p_curr.right = child
```

7.8 BST Inventory Assignment

From the course case study. Manage products using BST.

```
# products.csv format: product_id,product_name,price
# 1001,Widget A,9.99
# Skip headers and empty lines

# Each BST node: key=product_id, value=(product_name, price)

# 1. Read CSV, insert into BST
# 2. search_by_id(id) -> return (name, price) or None
# 3. remove(id) -> handle 3 deletion cases
# 4. inorder_traversal() -> show products sorted by id
# 5. Answer: average-case O(log n), worst-case O(n)
# 6. Five most expensive? Traverse and keep top 5
```

PART 8 - GRAPHS

Final exam: Graphs = 10 points bonus (no coding).

8.1 Graph Basics

A **graph** = nodes (vertices) connected by **edges**.

Trees are a special type of graph (connected, no cycles).

```
# Graph: A -- B
# | |
# C -- D
graph = {
"A": ["B", "C"],
"B": ["A", "D"],
"C": ["A", "D"],
"D": ["B", "C"]
}
```

8.2 BFS (Queue) - level by level

```
from collections import deque
def bfs(graph, start):
visited = set()
queue = deque([start])
while queue:
node = queue.popleft()
if node not in visited:
visited.add(node)
print(node, end=" ")
for nb in graph[node]:
if nb not in visited:
queue.append(nb)
```

8.3 DFS (Stack/Recursion) - go deep first

```
def dfs(graph, start, visited=None):
if visited is None:
visited = set()
visited.add(start)
print(start, end=" ")
for nb in graph[start]:
if nb not in visited:
dfs(graph, nb, visited)
```

PART 9 - RECURSION

A function that calls **itself**. Heavily used in trees and BSTs.

9.1 Every Recursive Function Has:

```
def solve(problem):
# 1. BASE CASE - when to STOP
if problem is smallest:
return answer
# 2. RECURSIVE CASE - call with smaller problem
return combine(problem, solve(smaller_problem))
```

9.2 Factorial

```
def factorial(n):
if n <= 1: return 1 # BASE
return n * factorial(n-1) # RECURSIVE

# fact(4) = 4 * fact(3)
# = 4 * 3 * fact(2)
# = 4 * 3 * 2 * fact(1)
# = 4 * 3 * 2 * 1 = 24
```

9.3 Fibonacci

```
def fib(n):
if n <= 1: return n # BASE
return fib(n-1) + fib(n-2) # RECURSIVE

# WARNING: fib(40) calls itself ~330 million times! O(2^n)

# ITERATIVE is much better:
def fib_iter(n):
if n <= 1: return n
a, b = 0, 1
for _ in range(2, n+1):
a, b = b, a + b
return b
```

9.4 Recursion on Trees (template)

```
def tree_func(root):
if root is None: # BASE
return something
left = tree_func(root.left) # RECURSE
right = tree_func(root.right) # RECURSE
return combine(root, left, right)

# Examples: height, sum nodes, search, count
```

PART 10 - ASSIGNMENTS

These are from the course emails and PDFs.

10.1 Student Data Linked List (svssvsdv.pdf)

File format: **id/name/age/gpa/major**

Example: **14752/Ghita/19/3.4/cs**

Q1: Append from file

```
def append_from_file(ll, file_handler):
for line in file_handler:
parts = line.strip().split('/')
if len(parts) >= 5:
s = (int(parts[0]), parts[1], int(parts[2]),
float(parts[3]), parts[4])
ll.append(s)
# File is ALREADY OPEN. Do NOT open it!
```

Q2: Display (id, name)

```
def display_students(ll):
curr = ll.head
while curr:
s = curr.data
print(f"({s[0]}, {s[1]})")
curr = curr.next
```

Q3: Remove by ID

```
def remove_by_id(ll, target_id):
curr, prev = ll.head, None
while curr:
if curr.data[0] == target_id:
if prev is None:
ll.head = curr.next
else:
prev.next = curr.next
if curr == ll.tail:
ll.tail = prev
return True
prev, curr = curr, curr.next
return False
```

Q4: Search by Major

```
def search_by_major(ll, major):
names = []
curr = ll.head
while curr:
if curr.data[4] == major:
names.append(curr.data[1])
curr = curr.next
return names
```

10.2 Queues & Stacks Assignment

4 exercises from the assignment PDF:

```
# Ex1: Reverse Queue Using Stack (see Part 6.5)
# Ex2: Filter Odd Numbers from Queue (see Part 6.6)
# Ex3: Merge Two Stacks Alternately (see Part 6.8)
# Ex4: Merge Two Sorted Queues (see Part 6.7)
```

10.3 BST Inventory Assignment

Managing products with BST. Key details from the email:

```
# Read products.csv
# product_id = BST key
# (product_name, price) = BST value
#
# Functions:
# - search_by_id(id) -> (name, price) or None
# - remove(id)
# - inorder_traversal()
```

```
#
# Performance questions:
# - Avg case: O(log n)
# - Worst case: O(n) (skewed tree)
# - Fix: use AVL or Red-Black tree (self-balancing)
# - 5 most expensive: traverse and find top 5
```

PART 11 - PRACTICE PROBLEMS

11.1 BST Recursion Problems

Find level of a node:

```
def find_level(root, target, level=1):
if root is None: return -1
if root.value == target: return level
l = find_level(root.left, target, level+1)
if l != -1: return l
return find_level(root.right, target, level+1)
```

Minimum depth:

```
def min_depth(root):
if root is None: return 0
if not root.left: return 1 + min_depth(root.right)
if not root.right: return 1 + min_depth(root.left)
return 1 + min(min_depth(root.left), min_depth(root.right))
```

Right view:

```
def right_view(root):
result = []
def dfs(n, d):
if not n: return
if d == len(result): result.append(n.value)
dfs(n.right, d+1)
dfs(n.left, d+1)
dfs(root, 0)
return result
```

Max path sum (root to leaf):

```
def max_path_sum(root):
if root is None: return float('-inf')
if not root.left and not root.right: return root.value
return root.value + max(max_path_sum(root.left), max_path_sum(root.right))
```

Lowest Common Ancestor:

```
def lca(root, p, q):
if root is None: return None
if root.value == p or root.value == q: return root.value
L = lca(root.left, p, q)
R = lca(root.right, p, q)
if L and R: return root.value
return L or R
```

Symmetric tree check:

```
def is_symmetric(root):
def check(l, r):
if not l and not r: return True
if not l or not r: return False
return (l.value == r.value and
check(l.left, r.right) and
check(l.right, r.left))
return check(root.left, root.right) if root else True
```

Sum of left leaves:

```
def sum_left_leaves(root):
def dfs(n, left):
if not n: return 0
if not n.left and not n.left and left:
return n.value
return dfs(n.left, True) + dfs(n.right, False)
return dfs(root, False)
```

Identical trees:

```
def is_identical(t1, t2):
if not t1 and not t2: return True
if not t1 or not t2: return False
```

```
return (t1.value == t2.value and
is_identical(t1.left, t2.left) and
is_identical(t1.right, t2.right))
```

11.2 Python Practice

Rotate list right by k:

```
def rotate(nums, k):
k = k % len(nums)
return nums[-k:] + nums[:-k]

# [1,2,3,4,5], k=2 -> [4,5,1,2,3]
```

Remove duplicates (maintain order):

```
def remove_duplicates(nums):
seen = set()
result = []
for n in nums:
if n not in seen:
seen.add(n)
result.append(n)
return result

# [1,2,2,3,4,4,5] -> [1,2,3,4,5]
```

Count frequencies with dict:

```
def count_freq(nums):
counts = {}
for n in nums:
if n in counts:
counts[n] += 1
else:
counts[n] = 1
return counts

# [2,2,4,5,4,4,5,7] -> {2:2, 4:3, 5:2, 7:1}
```

Max product pair:

```
def max_product_pair(t):
s = sorted(t)
a, b = s[-2], s[-1]
c, d = s[0], s[1]
return (a, b) if a*b > c*d else (c, d)

# (1, 10, 3, 7, -5, 6) -> (10, 7)
```

Check magic square:

```
def is_magic(m):
n = len(m)
t = sum(m[0])
if any(sum(r) != t for r in m): return False
if any(sum(m[i][j] for i in range(n)) != t for j in range(n)): return False
if sum(m[i][i] for i in range(n)) != t: return False
if sum(m[i][n-1-i] for i in range(n)) != t: return False
return True
```

PART 12 - QUICK REFERENCE

Big O Cheat Sheet

```
O(1) array[i], push/pop, peek
O(log n) binary search, BST balanced
O(n) loop, linear search
O(n log n) merge sort, quick sort
O(n^2) bubble sort, nested loops
```

Stack vs Queue

```
STACK (LIFO) QUEUE (FIFO)
push(value) enqueue(value)
pop() dequeue()
peek() front()
plate stack waiting line
bracket check airport security
```

BST Complexity

```
Operation Average Worst (skewed)
Search O(log n) O(n)
Insert O(log n) O(n)
Delete O(log n) O(n)
Inorder O(n) O(n)
```

Common Exam Mistakes

Using = instead of == in if
Forgetting to close files (use 'with')
Off-by-one (range(n) = 0 to n-1)
Modifying list while iterating
No base case in recursion
Opening a file that is already open
Creating object already passed as parameter
Using self in regular function
Forgetting to update tail in linked list
Forgetting empty case (list/stack/queue)

How to Start Any Coding Problem

1. Read twice: what am I GIVEN? what must I RETURN?
2. Try small example by hand
3. What SHAPE? (search? sort? traverse? count?)
4. Pick the data structure
5. Write steps in plain English
6. Code one step at a time
7. Check edge cases: empty, one item, all same

Exam Day Checklist

- Two pencils + eraser
- AUI ID on desk
- Phone OFF in bag
- Read instructions carefully
- Descriptive names (gpa, not x)
- Use scratch paper first
- Put exam inside scratch paper before submitting

END OF GUIDE - Good luck on your exams!

Al Akhawayn University in Ifrane - School of Science and Engineering

Based on the CSC2302 course materials